

Replicação de Banco de Dados no MySQL

Guilherme Leite
Deyvidy Alves
Tayson Silva

Sumário

1	Introdução	3
2	Fundamentação Teórica	3
3	Configuração do Ambiente	4
4	Testes Realizados	8
5	Resultados Obtidos	11
	Referências	11

1 Introdução

Os objetivos deste texto são: i) explicar o que é replicação de banco de dados e tópicos relacionados à replicação; ii) apresentar a configuração de um ambiente para replicação de banco de dados no MySQL; e iii) documentar os resultados obtidos.

2 Fundamentação Teórica

A replicação de banco de dados é uma técnica que permite que dados de um banco de dados – conhecido como origem, outrora tratado por *master* (mestre) – sejam copiados por um ou mais servidores de banco de dados, que são as réplicas. A depender da configuração, é possível replicar todos os banco de dados, apenas alguns banco de dados ou mesmo selecionar tabelas dentro de um banco de dados para serem replicadas.

No MySQL, a replicação funciona da seguinte forma: o banco de dados de origem escreve todas as alterações feitas num arquivo chamado *binary log*; uma vez que a réplica é inicializada, são criados duas linhas de processo, a saber, a *IO Thread*, que é quem se conecta ao banco de dados de origem, lê o conteúdo do *binary log* e o copia em um arquivo próprio do servidor da réplica chamado *relay log*, e a *SQL Thread*, que é quem lê os eventos contidos no *relay log* e, tão logo quanto possível, aplica as alterações na réplica.

Há dois métodos na replicação de dados. A diferença nesses métodos tem a ver sobre como as réplicas rastreiam os eventos do banco de dados de origem que já foram processados por elas.

O primeiro método, que é o tradicional, é a replicação baseada na posição do *binary log*. Quando você promove uma instância do MySQL a uma réplica usando esse método, você deve prover-lhe as coordenadas precisas do arquivo do *binary log*. O *binary log* é como se fosse um livro, e cada alteração feita na origem é gravada nesse livro, e é esse livro que é transmitido às réplicas. Mas veja, as réplicas recebem todo o livro, se elas tiverem já processado os eventos narrados até o capítulo 3, e eu não especificar que ela deve continuar a replicação a partir do capítulo 4, ela vai ler o conteúdo do livro desde o início, porque ela recebe o livro inteiro. Então, nesse método, eu preciso especificar-lhe a partir de qual posição no arquivo ela vai lê-lo. O problema é que, se a origem morrer e uma réplica for promovida a origem, o livro de alterações dela (seu *binary log*), vai ser de uma edição diferente e, quando uma réplica disser a essa nova origem “olha, eu parei na posição x do *binary log* y”, pode dar problema, posto que essa posição no novo arquivo de *binary log* pode já ter sido processada ou, pior, pode nem existir ainda. E, por isso, sempre que algo assim acontecer, é necessário ir em cada réplica e, manualmente, escrever-lhes as coordenadas do novo *binary log*.

Por isso, surgiu o método que utiliza GTIDs (*Global Transaction Identifiers*) nas replicações. Esse método é bem simples. Veja, assim como em obras de numeração universal, como na Bíblia, onde a referência “capítulo ‘x’, versículo ‘y’” (ou a combinação de suras e ayats no caso do Alcorão) aponta, *mutatis mutandis*, para o mesmo texto, assim o método com GTIDs identifica universalmente cada transação com o UUID do servidor de origem e um número inteiro positivo de sequência, pois cada alteração é carimbada com um identificador único reconhecido por todos os servidores que participam desta topologia. Ele é um método de identificação universal e absoluto, e não particular e relativo como no primeiro. Assim, a réplica pode dizer para qualquer servidor promovido a origem que leu

até a transação *n*, e eles irão entender e vão enviar-lhe apenas as alterações que ela não leu.

Por padrão, a replicação no MySQL é assíncrona. Isso quer dizer que a origem não sabe se ou quando a réplica pegou e processou as transações copiadas, ou seja, não há uma garantia de que qualquer transação tenha chegado a qualquer réplica. Já na replicação semi síncrona, a origem espera que pelo menos uma réplica tenha recebido e gravado a transação no seu *relay log* antes de confirmar a transação localmente. Ou seja, se o servidor de origem cair, teremos que os dados existem em pelo menos dois servidores, a saber, no servidor que caiu e em uma das réplicas.

A replicação é muito valiosa porque, ao distribuir a carga de leitura entre várias réplicas, o servidor de origem fica dedicado apenas às escritas e atualizações, é aumentada bastante a velocidade de leitura; além disso, ela possibilita a segurança de dados, pois eu posso parar o processo de replicação na réplica e fazer processos de *backups* nela; eu também posso fazer as análises pesadas e relatórios de dados nas réplicas, evitando de sobrecarregar o servidor de origem; e ela também facilita a cópia de dados para locais distantes, o que ajuda pessoas de locais distantes do *database* de origem a acessarem o banco de dados rapidamente. Tudo isso é muito útil em ambientes corporativos, pois as empresas precisam que seus dados sejam tolerantes a falhas e tenham alta disponibilidade, que tenham alcance maior e que não sofram lentidão.

Entretanto, a replicação também apresenta desafios e desvantagens, pois é difícil fazer o monitoramento e gerenciamento de várias réplicas, também é difícil manter a consistência de todos os dados em todas as réplicas, principalmente se for uma replicação assíncrona; além disso, a replicação também necessita de mais CPU, devido às *threads* na replicação e também de mais armazenamento devido aos arquivos de *binary log* e *relay log*.

A replicação é diferente de *backup* e de alta disponibilidade. O *backup* é um mecanismo que você usa para salvar seus dados, um processo que pode ser longo e demorado, ele serve para proteger os dados, por exemplo, de alguém fazer um DELETE sem WHERE. A replicação é a técnica que permite a sincronização automática de dados entre servidores de banco de dados. Ela não serve para tirar um retrato do banco de dados. Se alguém fizer um DELETE na origem, essa transação irá reverberar nas outras réplicas. Já a alta disponibilidade é a capacidade que um sistema tem de estar disponível em 99,99% do tempo, ela é um objetivo arquitetural para manter um sistema operacional o máximo de tempo possível durante todos os dias.

3 Configuração do Ambiente

Por praticidade, foram criados dois serviços MySQL num único arquivo `docker-compose.yml`:

```
mysql-master:
  image: mysql:8.4
  container_name: mysql-master
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: root
  ports:
    - "3306:3306"
  volumes:
```

```

    - master_data:/var/lib/mysql
    - ./master/my.cnf:/etc/mysql/conf.d/my.cnf
networks:
    - replication

mysql-slave:
    image: mysql:8.4
    container_name: mysql-slave
    restart: unless-stopped
    environment:
        MYSQL_ROOT_PASSWORD: root
    ports:
        - "3307:3306"
    volumes:
        - slave_data:/var/lib/mysql
        - ./slave/my.cnf:/etc/mysql/conf.d/my.cnf
    depends_on:
        - mysql-master
    networks:
        - replication

volumes:
    master_data:
    slave_data:

networks:
    replication:
        driver: bridge

```

Aqui eu defino a criação de dois serviços MySQL, um chamado `mysql-master` e outro chamado `mysql-slave`, que estão sob a mesma rede virtual `replication`. Também faço o segundo serviço depender da criação do primeiro para ser inicializado.

Mas, para permitir a replicação, eu preciso editar o arquivo `.cnf` de ambos os serviços. Veja o `my.cnf` do *master*:

```

[mysqld]
server-id = 1
bind-address = 0.0.0.0
log-bin=/var/lib/mysql/mysql-bin
gtid_mode = ON
enforce-gtid-consistency = ON

```

Server-id: serve para identificar o servidor na topologia. Ele tem de ser, por óbvio, único.

Bind-address = 0.0.0.0, significa que o MySQL vai aceitar conexões vindas de qualquer lugar, incluindo na rede docker, então isso permite que o meu *slave* saia do seu container e vá até o container do *master* para pegar o conteúdo do *binlog*.

log-bin=/var/lib/mysql/mysql-bin: basicamente eu estou ativando o conteúdo do *binary log*, então assim o MySQL vai começar a gravar as alterações no *binary log*.

gtid_mode = ON: basicamente, ativa o GTID (*Global Transaction Identifiers*).

enforce-gtid-consistency = ON: é uma variável de configuração global do MySQL que, quando ativada, impede que comandos que não sejam seguros para o GTID sejam executados.

OBS: também poderia colocar a linha `binlog_do_db = <nome do seu banco de dados>`. Isso faria com que apenas o banco de dados em questão fosse replicado; os outros que não estivessem nessa linha seriam, portanto, ignorados pelas réplicas.

E o `my.cnf` do *slave*:

```
server-id = 2
gtid_mode = ON
enforce_gtid_consistency = ON
log-bin=/var/lib/mysql/mysql-bin
relay-log=/var/lib/mysql/mysql-relay-bin
read_only = ON
skip-replica-start = ON
```

`relay-log=/var/lib/mysql/mysql-relay-bin` garante que o *relay log* funcione, é onde a *thread* IO grava as alterações que ela copiou.

O **read_only** serve para proteger o *slave* de escritas, embora ele não impeça a *SQL Thread* de aplicar as alterações.

Skip-replica-start: apenas impede que a réplica inicie o processo de replicação automaticamente.

Depois disso, nós criamos um usuário com o comando **CREATE USER** do MySQL para criar um usuário para réplica, é com ele que a réplica vai entrar em contato com a origem:

```
CREATE USER 'repl'@'%' IDENTIFIED BY '1234';
```

E então dei-lhe os privilégios que o MySQL recomenda nesse cenário:

```
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
```

Depois, no servidor da réplica, é executado o comando **CHANGE SOURCE TO** para fazer a conexão da réplica com a origem. Veja:

```
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST='mysql-master',
  SOURCE_PORT=3306,
  SOURCE_USER='repl',
  SOURCE_PASSWORD='1234',
  SOURCE_AUTO_POSITION=1,
  GET_SOURCE_PUBLIC_KEY=1;
```

SOURCE_HOST eu defino o endereço do *host*. Como estou usando docker, eu coloco o nome do serviço, se eu colocasse o IP, eu teria que modificar toda vez que o container reiniciasse;

SOURCE_PORT eu defino a porta de conexão;

SOURCE_USER eu passo o nome do usuário que foi criado na origem para estabelecer a conexão;

SOURCE_PASSWORD eu passo a senha desse usuário;

SOURCE_AUTO_POSITION = 1 eu digo à réplica para ela usar GTIDs para identificar a transação correta a ser copiada;

GET_SOURCE_PUBLIC_KEY = 1, ele serve para autenticação via chave pública (RSA), pois estou usando o sistema `caching_sha2_password`.

Executado esse comando, eu executo **START REPLICA** e o processo de replicação se inicia. Ao rodar o comando **SHOW REPLICA STATUS\G** pela primeira vez, é mostrado na tela:

```
mysql> SHOW REPLICA STATUS \G;
***** 1. row *****
      Replica_IO_State: Waiting for source to send event
      Source_Host: mysql-master
      Source_User: repl
      Source_Port: 3306
      Connect_Retry: 60
      Source_Log_File: mysql-bin.000003
      Read_Source_Log_Pos: 731
      Relay_Log_File: mysql-relay-bin.000003
      Relay_Log_Pos: 948
      Relay_Source_Log_File: mysql-bin.000003
      Replica_IO_Running: Yes
      Replica_SQL_Running: Yes
      Replicate_Do_DB:
      Replicate_Ignore_DB:
      Replicate_Do_Table:
      Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
      Replicate_Wild_Ignore_Table:
      Last_Errno: 0
      Last_Error:
      Skip_Counter: 0
      Exec_Source_Log_Pos: 731
      Relay_Log_Space: 2997750
      Until_Condition: None
      Until_Log_File:
      Until_Log_Pos: 0
      Source_SSL_Allowed: No
      Source_SSL_CA_File:
      Source_SSL_CA_Path:
      Source_SSL_Cert:
      Source_SSL_Cipher:
      Source_SSL_Key:
      Seconds_Behind_Source: 0
      Source_SSL_Verify_Server_Cert: No
      Last_IO_Errno: 0
      Last_IO_Error:
      Last_SQL_Errno: 0
```

```

        Last_SQL_Error:
Replicate_Ignore_Server_Ids:
        Source_Server_Id: 1
            Source_UUID: d4776cc4-99f0-11f1-98a0-d637b6a67963
        Source_Info_File: mysql.slave_master_info
            SQL_Delay: 0
        SQL_Remaining_Delay: NULL
Replica_SQL_Running_State: Replica has read all relay log; waiting for more
updates
        Source_Retry_Count: 10
            Source_Bind:
        Last_IO_Error_Timestamp:
Last_SQL_Error_Timestamp:
        Source_SSL_Crl:
        Source_SSL_Crlpath:
Retrieved_Gtid_Set: d4776cc4-99f0-11f1-98a0-d637b6a67963:1-7
Executed_Gtid_Set: d4776cc4-99f0-11f1-98a0-d637b6a67963:1-7,
                    d483a3ce-99f0-11f1-9dbc-faaf9f75c3d9:1-5
        Auto_Position: 1
Replicate_Rewrite_DB:
        Channel_Name:
        Source_TLS_Version:
Source_public_key_path:
Get_Source_public_key: 1
        Network_Namespace:
1 row in set (0.00 sec)

ERROR:
No query specified

```

Os dois indicadores mais importantes são:

```

Replica_IO_Running: Yes
Replica_SQL_Running: Yes

```

Sim, esses dois se referem às *threads* faladas anteriormente. O primeiro diz que a réplica está conseguindo ler o conteúdo do *binary log* e gravá-lo no *relay log*; e o segundo diz que a *SQL Thread* está conseguindo ler o conteúdo do *relay log* e aplicar no banco de dados da réplica.

4 Testes Realizados

Foram executados os seguintes comandos no MySQL da origem:

```

CREATE DATABASE teste;
USE teste;

CREATE TABLE colaboradores (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    cargo VARCHAR(50),

```



```

    data_admissao DATE
);

INSERT INTO colaboradores (nome, cargo, data_admissao)
VALUES ('Guilherme Oliveira', 'Analista de Sistemas', '2023-01-15'),
      ('Maria Silva', 'Desenvolvedora', '2023-03-20');

UPDATE colaboradores
SET cargo = 'Engenheiro de Software'
WHERE nome = 'Guilherme Oliveira';

DELETE FROM colaboradores WHERE id = 2;

CREATE INDEX idx_nome ON colaboradores(nome);

ALTER TABLE colaboradores ADD COLUMN salario DECIMAL(10,2);

```

Ao consultar o conteúdo do arquivo do *binary log*, podemos ver que as alterações estão de fato sendo gravadas, veja:

```

mysql> show binary log status;
+-----+-----+-----+-----+-----+
| File | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000003 | 3199 | | | d4776cc4-99f0-11f1-98a0-d637b6a67963:1-16 |
+-----+-----+-----+-----+-----+

1 row in set (0.00 sec)

mysql> show binlog events in 'mysql-bin.000003' limit 20;
+-----+-----+-----+-----+-----+-----+
| Log_name | Pos | Event_type | Server_id | End_log_pos | Info |
+-----+-----+-----+-----+-----+-----+
| mysql-bin.000003 | 4 | Format_desc | 1 | 127 | Server ver: 8.4.11, Binlog ver: 4 |
| mysql-bin.000003 | 127 | Previous_gtid | 1 | 198 | d4776cc4-99f0-11f1-98a0-d637b6a67963:1-5 |
| mysql-bin.000003 | 198 | Gtid | 1 | 277 | SET @@SESSION.GTID_NEXT= '...:6' |
| mysql-bin.000003 | 277 | Query | 1 | 511 | CREATE USER 'repl'@'%' IDENTIFIED WITH ... |
| mysql-bin.000003 | 511 | Gtid | 1 | 588 | SET @@SESSION.GTID_NEXT= '...:7' |
| mysql-bin.000003 | 588 | Query | 1 | 731 | GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%' /* xid=10 */ |
| mysql-bin.000003 | 731 | Gtid | 1 | 808 | SET @@SESSION.GTID_NEXT= '...:8' |
| mysql-bin.000003 | 808 | Query | 1 | 943 | create database teste_replica /* xid=24 */ |
| mysql-bin.000003 | 943 | Gtid | 1 | 1020 | SET @@SESSION.GTID_NEXT= '...:9' |
| mysql-bin.000003 | 1020 | Query | 1 | 1151 | drop database teste_replica /*

```

```

      xid=46 */ |
| mysql-bin.000003 | 1151 | Gtid | 1 | 1228 | SET @@SESSION.GTID_NEXT= '...:10'
|
| mysql-bin.000003 | 1228 | Query | 1 | 1339 | CREATE DATABASE teste /* xid=51
*/ |
| mysql-bin.000003 | 1339 | Gtid | 1 | 1418 | SET @@SESSION.GTID_NEXT= '...:11'
|
| mysql-bin.000003 | 1418 | Query | 1 | 1655 | use 'teste'; CREATE TABLE
colaboradores (...) /* xid=56 */ |
| mysql-bin.000003 | 1655 | Gtid | 1 | 1734 | SET @@SESSION.GTID_NEXT= '...:12'
|
| mysql-bin.000003 | 1734 | Query | 1 | 1810 | BEGIN |
| mysql-bin.000003 | 1810 | Table_map | 1 | 1882 | table_id: 93 (teste.
colaboradores) |
| mysql-bin.000003 | 1882 | Write_rows | 1 | 2002 | table_id: 93 flags:
STMT_END_F |
| mysql-bin.000003 | 2002 | Xid | 1 | 2033 | COMMIT /* xid=57 */ |
| mysql-bin.000003 | 2033 | Gtid | 1 | 2112 | SET @@SESSION.GTID_NEXT= '...:13'
|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
20 rows in set (0.00 sec)

```

E o `show replica status\G` (executado no servidor da réplica), mostra o seguinte:

```

mysql> show replica status\G;
***** 1. row *****
      Replica_IO_State: Waiting for source to send event
      Source_Host: mysql-master
      Source_User: repl
      Source_Port: 3306
      Connect_Retry: 60
      Source_Log_File: mysql-bin.000003
      Read_Source_Log_Pos: 3199
      Relay_Log_File: mysql-relay-bin.000003
      Relay_Log_Pos: 3416
      Relay_Source_Log_File: mysql-bin.000003
      Replica_IO_Running: Yes
      Replica_SQL_Running: Yes
      Exec_Source_Log_Pos: 3199
      Relay_Log_Space: 3000218
      Seconds_Behind_Source: 0
      Source_Server_Id: 1
      Source_UUID: d4776cc4-99f0-11f1-98a0-d637b6a67963
      Retrieved_Gtid_Set: d4776cc4-99f0-11f1-98a0-d637b6a67963:1-16
      Executed_Gtid_Set: d4776cc4-99f0-11f1-98a0-d637b6a67963:1-16,
                        d483a3ce-99f0-11f1-9dbc-faaf9f75c3d9:1-5
      Auto_Position: 1
1 row in set (0.00 sec)

ERROR:
No query specified

```

5 Resultados Obtidos

Com o teste, gravado em vídeo, foi possível perceber a réplica funcionando corretamente. Foi possível ver o *status* da réplica e ver também o conteúdo do arquivo do *binary log*. Também foi possível perceber que a configuração está usando *Row-Based Replication* (RBR), comprovado pela presença do evento `Write_rows`, que é muito mais seguro já que ele envia uma espécie de *snapshot* da linha alterada. E também foi possível ver que essa configuração de replicação no MySQL é tolerante a falhas, já que a origem continuou gravando seus dados no *binary log* mesmo quando a replicação estava desativada no `mysql-slave` e, ao voltar ao funcionamento, a réplica baixou todas as alterações que foram feitas quando estava sem funcionar; nisso foi visto o funcionamento do GTID, já que eu não precisei colocar manualmente as coordenadas do arquivo *binary log*.

Referências

DRAKE, Mark; KURUP, Manikandan. **How To Set Up Replication in MySQL**. [S. l.]: DigitalOcean, 15 ago. 2026. Disponível em: <https://www.digitalocean.com/community/tutorials/how-to-set-up-replication-in-mysql>. Acesso em: 16 ago. 2026.

FIVETRAN. **MySQL replication: data replication types and setup methods explained**. [S. l.]: Fivetran, 15 ago. 2023. Disponível em: <https://www.fivetran.com/learn/mysql-database-replication>. Acesso em: 18 ago. 2026.

ORACLE. **MySQL 8.0 Reference Manual: including MySQL NDB Cluster 8.0**. [S. l.]: Oracle Corporation, 8 jul. 2026. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 19 ago. 2026.

ŠUŠTER, Ivan et al. **Database replication: ensuring MySQL data integrity**. Quaestus Multidisciplinary Research Journal.